

TEMAT 13 - JASKINIA

SZYMON LOTTIG

NR ALBUMU: 155286

LINK DO REPOZYTORIUM: [GITHUB](#)

1. Założenia projektowe i opis działania

Opis zadania:

Celem projektu było napisanie symulacji obsługi ruchu turystycznego w jaskini. Zgodnie z tematem, system obsługuje dwie trasy zwiedzania o różnych limitach miejsc oraz kładkę wejściową, która stanowi wąskie gardło (ruch odbywa się wahadłowo - jednokierunkowo). Całość musiała zostać zrealizowana na systemie Linux przy użyciu języka C++ oraz mechanizmów komunikacji międzyprocesowej (IPC).

Struktura programu oraz opis plików źródłowych:

Symulacja składa się z kilku niezależnych programów, które komunikują się ze sobą:

- **main.cpp (program init)**
Narzędzie uruchamiane na początku. Tworzy zasoby systemowe (pamięć dzielona, semaforey, kolejki komunikatów) i ustawia ich wartości początkowe (np. limity miejsc).
- **przewodnik.cpp (program przewodnik)**
Pełni rolę serwera/monitora. Inicjuje dostęp do pamięci dzielonej, wyświetla aktualny stan jaskini (liczba osób, status tras) i reaguje na sygnały zamykające trasy.
- **kasjer.cpp (program kasjer)**
Odpowiada za logikę sprzedaży biletów. Odbiera komunikaty od klientów, nalicza statystyki sprzedaży i weryfikuje wiek (np. kieruje seniorów >75 lat tylko na Trasę 2).
- **klient.cpp (program klient)**
Generator ruchu. Tworzy procesy potomne (turystów) za pomocą funkcji fork(). Każdy proces-turysta żyje własnym życiem: samodzielnie kupuje bilet, czeka na semaforze (wejście na kładkę) i zwiedza jaskinię.
- **straznik.cpp (program straznik)**
Panel sterowania. Pozwala użytkownikowi ręcznie wysyłać sygnały do procesu Przewodnika w celu wymuszenia zamknięcia lub otwarcia tras.
- **cleanup.cpp (program clean)**
Program sprzątający, który usuwa wszystkie zasoby systemowe (IPCS) po zakończeniu symulacji
- **tools.cpp / common.h**
Biblioteka funkcji pomocniczych i pliki nagłówkowe ze strukturami danych.

Zrealizowane funkcjonalności i elementy dodatkowe

Udało się zaimplementować wszystkie wymagane mechanizmy. Synchronizacja dostępu do kładki działa poprawnie dzięki semaforom (zapobieganie zakleszczeniom przy zmianie kierunku). Zrealizowano obsługę priorytetów w kolejce komunikatów (osoby powracające są obsługiwane poza kolejnością). Jako element dodatkowy dodano **kolorowanie wyjścia** w terminalu (**kody ANSI**), co znacznie poprawia czytelność logów i ułatwia monitorowanie stanu tras.

2. Napotkane problemy i rozwiązania

- Zakleszczenia na kładce (Deadlock)

Przy równoczesnym ruchu w obu kierunkach dochodziło do blokady mostu.

Rozwiązanie:

Wymuszony ruch jednokierunkowy w funkcji `czekaj_na_most` (`klient.cpp`). Proces może wejść tylko, gdy kierunek jest zgodny lub most pusty.

- Synchronizacja zamykania tras

Pojawił się problem z procesami, które w momencie interwencji Strażnika znajdowały się już na kładce. Wchodziły one na trasę mimo jej zamknięcia, ponieważ sprawdzały status otwarcia zbyt wcześnie (przed wejściem na most).

Rozwiązanie:

Wdrożono obsługę sygnałów w Przewodniku oraz przesunięto weryfikację stanu trasy w procesie Klienta na moment po zejściu z kładki, co pozwala na natychmiastowe wycofanie się turysty w razie blokady.

- Liczniki osób przekłamywały rzeczywistość przy dużej liczbie procesów.

Rozwiązanie:

Użyto semafora nr 0 jako Mutexu (`lock_sem` w `tools.cpp`) do blokowania dostępu do pamięci dzielonej podczas modyfikacji zmiennych.

- Procesy zombie i niezwolnione zasoby

Po przerwaniu symulacji w systemie zostawały aktywne procesy i zajęta pamięć.

Rozwiązanie:

Dodano funkcję `wait()` w generatorze turystów oraz napisano program `clean`, który usuwa wszystkie zasoby IPC (pamięć, kolejki, semaforey).

3. Testy

- **Weryfikacja przepustowości kładki**

Cel : Sprawdzenie, czy semafor poprawnie blokuje nadmiarowych turystów przed wejściem na most.

Wynik: Licznik osób na kładce w panelu Przewodnika zmieniał się dynamicznie, ale nigdy nie przekroczył wartości 3, mimo dużej liczby oczekujących procesów.

Dowód działania:

```
STATUS JASKINI (PID: 1721)
  Stan otwarcia: OTWARTA
  Trasa 1: CZYNNA
  Trasa 2: CZYNNA
-----
  BILANS: Sprzedane: 1207 | Darmowe (dzieci <3): 40
-----
  Ruch w srodku:
  -> Kładka (Wejscie): 3 / 3 osob | Kierunek: <<< (WYCHODZA)
  -> Trasa 1: 3 / 10 osob
  -> Trasa 2: 8 / 15 osob
-----
Sterowanie: ./straznik
```

- **Obsługa priorytetów (Bilet dla powracającego)**

Cel : Weryfikacja, czy osoby ze statusem "powracający" są obsługiwane poza kolejnością.

Wynik: Kasjer poprawnie identyfikował te bilety (typ komunikatu 2) i pobierał je z kolejki przed zwykłymi biletami (typ 1).

Dowód działania:

```
[KASJER] [PRIORYTET - OMINAL KOLEJKE!] Klient ID: 576 (Wiek: 49, Trasa: 1)
  Bilet ZNIZKOWY (-50% dla powracajacego).
RAZEM: Sprzedano: 1141 | Darmowe wejscia: 36

[KASJER] [ZWYKLY] Klient ID: 577 (Wiek: 8, Trasa: 2)
  Bilet SPRZEDANY (Cena normalna).
RAZEM: Sprzedano: 1142 | Darmowe wejscia: 36

[KASJER] [ZWYKLY] Klient ID: 578 (Wiek: 45, Trasa: 1)
  Bilet SPRZEDANY (Cena normalna).
RAZEM: Sprzedano: 1143 | Darmowe wejscia: 36

[KASJER] [PRIORYTET - OMINAL KOLEJKE!] Klient ID: 579 (Wiek: 69, Trasa: 2)
  Bilet ZNIZKOWY (-50% dla powracajacego).
RAZEM: Sprzedano: 1144 | Darmowe wejscia: 36
```

- **Selekcja wiekowa (Seniorzy i Dzieci)**

Cel: Sprawdzenie zgodności z regulaminem (dzieci < 8 lat i seniorzy > 75 lat tylko na Trasę 2).

Wynik: Wszyscy wylosowani seniorzy oraz małe dzieci otrzymali przydział na Trasę 2. Nie odnotowano błędnych przydziałów na Trasę 1.

Dowód działania:

```
[KASJER] [ZWYKLY] Klient ID: 585 (Wiek: 2, Trasa: 2)
      Wstęp WOLNY (Dziecko < 3 lat).
RAZEM: Sprzedano: 1149 | Darmowe wejścia: 37

[KASJER] [ZWYKLY] Klient ID: 586 (Wiek: 79, Trasa: 2)
      Bilet SPRZEDANY (Cena normalna).
RAZEM: Sprzedano: 1150 | Darmowe wejścia: 37
```

- **Zamykanie trasy przez Strażnika (Sygnały)**

Cel: Sprawdzenie reakcji systemu na interwencję operatora.

Wynik: Po wysłaniu sygnału SIGUSR1 przez Strażnika, Przewodnik zamknął trasę, a nowe procesy próbujące wejść otrzymały odmowę i zakończyły działanie.

Dowód działania:

```
STATUS JASKINI (PID: 1721)
Stan otwarcia: OTWARTA
Trasa 1: ZAMKNIETA [X]
Trasa 2: CZYNNA
-----
BILANS: Sprzedane: 1858 | Darmowe (dzieci <3): 59
-----
Ruch w srodtku:
-> Kładka (Wejście): 2 / 3 osob | Kierunek: >>> (WCHODZA)
-> Trasa 1: 0 / 10 osob
-> Trasa 2: 15 / 15 osob
-----
Sterowanie: ./straznik
```

```
(PID: 4249) >> Wszedłem na KLADKE. Ide do trasy 1
(PID: 4250) >> Wszedłem na KLADKE. Ide do trasy 1
(PID: 4249) [!] Trasa 1 ZAMKNIETA przez Straznika! Powrot.
(PID: 4251) >> Wszedłem na KLADKE. Ide do trasy 1
(PID: 4250) [!] Trasa 1 ZAMKNIETA przez Straznika! Powrot.
(PID: 4252) >> Wszedłem na KLADKE. Ide do trasy 1
(PID: 4251) [!] Trasa 1 ZAMKNIETA przez Straznika! Powrot.
(PID: 4252) [!] Trasa 1 ZAMKNIETA przez Straznika! Powrot.
```

4. Instrukcja kompilacji i uruchomienia

- Kompilacja

```
g++ -o init main.cpp tools.cpp
g++ -o przewodnik przewodnik.cpp tools.cpp
g++ -o kasjer kasjer.cpp tools.cpp
g++ -o straznik straznik.cpp tools.cpp
g++ -o klient klient.cpp tools.cpp
g++ -o clean cleanup.cpp tools.cpp
```

- Uruchomienie (W osobnych terminalach)

1. ./init (inicjalizacja zasobów)
2. ./przewodnik (monitoring - wyświetla PID potrzebny dla strażnika)
3. ./kasjer (obsługa biletów)
4. ./straznik (panel sterowania - Tutaj wpisujemy PID z przewodnika)
5. ./klient (rozpoczęcie symulacji)

- Zakończenie symulacji

Po zakończeniu testów należy uruchomić ./clean, aby usunąć zasoby systemowe.

5. Implementacja - Linki do kodów

a) Tworzenie i obsługa procesów (fork, exit, wait)

- fork()

Wykorzystywana w module generatora ruchu (klient.cpp). Wewnątrz pętli głównej proces macierzysty wywołuje fork(), tworząc procesy potomne symulujące pojedynczych turystów. Każdy potomek otrzymuje kopię pamięci rodzica, ale wykonuje niezależną ścieżkę kodu (tulist_life).

LINK: [fork\(\)](#)

- exit()

Służy do bezpiecznego zakończenia procesu turysty po zwiedzeniu jaskini (lub rezygnacji) i odłączeniu pamięci dzielonej. Zwraca kod wyjścia 0 do systemu.

LINK: [exit\(\)](#)

- wait()

Użyta w generatorze, aby oczekiwać na zakończenie procesów potomnych. Zapobiega to powstawaniu tzw. procesów zombie, które blokowałyby tablicę procesów w systemie.

LINK: [wait\(\)](#)

b) Obsługa sygnałów (signal, kill)

- **signal()**
Użyta w procesie przewodnik.cpp do zarejestrowania własnych funkcji obsługi dla sygnałów SIGUSR1 oraz SIGUSR2. Dzięki temu proces nie jest ubijany przez te sygnały, lecz wykonuje logikę zamknięcia/otwarcia trasy.
LINK: [signal\(\)](#)
- **kill()**
Wykorzystana w panelu sterowania (straznik.cpp). Pozwala użytkownikowi wysłać sygnał do procesu Przewodnika (o wskazanym PID), co symuluje interwencję obsługi technicznej jaskini.
LINK: [kill\(\)](#)

c) Segmenty pamięci dzielonej (shmget, shmat)

- **shmget()**
Służy do utworzenia segmentu pamięci współdzielonej w systemie, identyfikowanego unikalnym kluczem. Pamięć ta przechowuje strukturę CaveState ze stanem całej jaskini.
LINK: [shmget\(\)](#)
- **shmat()**
Przyłącza utworzony segment pamięci do przestrzeni adresowej procesu. Dzięki temu procesy Przewodnika, Kasjera i Klienta mogą operować na tych samych zmiennych (licznikach osób).
LINK: [shmat\(\)](#)

d) Synchronizacja procesów (semop, semget)

- **semop()**
Kluczowa funkcja systemowa służąca do operacji na semaforach. W projekcie została opakowana w funkcje pomocnicze lock_sem (operacja -1) i unlock_sem (operacja +1) w pliku tools.cpp. Służy do realizacji sekcji krytycznych (ochrona pamięci dzielonej) oraz wstrzymywania turystów przed kładką.
LINK: [semop\(\)](#)

e) Kolejki komunikatów (msgsnd, msgrcv)

- **msgsnd()**

Wysyła strukturę z danymi turysty (wiek, typ biletu) do kolejki komunikatów. Używana przez proces klienta do "ustawienia się w kolejce" do kasy.

LINK: [msgsnd\(\)](#)

- **msgrcv()**

Odbiera komunikaty z kolejki. Zastosowanie flagi IPC_NOWAIT oraz selektywnego odbierania (typ komunikatu 2) pozwala Kasjerowi w pierwszej kolejności obsługiwać klientów priorytetowych (powracających), co realizuje wymóg obsługi priorytetów.

LINK: [msgrcv\(\)](#)